00 — PyTorch as a Numerical Language

How tensors store numbers, how broadcasting lines up shapes, and how autograd computes gradients, told through one small function you will reuse throughout the course.

Reading time: about 35 minutes. You need: matrices, derivatives and basic Python. You do not need: any prior PyTorch.

A bug that does nothing

Here is a small piece of attention code. Three tokens score each other, and we want every token to pay less attention to token 1. So we add a bias of −5 to token 1's column before the softmax.

```
scores = torch.zeros(3, 3)                  # 3 queries x 3 keys
key_bias = torch.tensor([0., -5., 0.])      # discourage key 1

good = torch.softmax(scores + key_bias, dim=-1)
bad = torch.softmax(scores + key_bias.reshape(3, 1), dim=-1)
```

The two versions differ by a single `reshape`. Here are the printed values, side by side:

```
good                                bad
[[0.4983, 0.0034, 0.4983],          [[0.3333, 0.3333, 0.3333],
 [0.4983, 0.0034, 0.4983],           [0.3333, 0.3333, 0.3333],
 [0.4983, 0.0034, 0.4983]]           [0.3333, 0.3333, 0.3333]]
```

The good version works: attention to token 1 drops to 0.3%. The bad version gives exactly the same result as having no bias at all. It raised no error and printed no warning. The bias silently did nothing.

What went wrong? `reshape(3, 1)` turned the bias into a column, so PyTorch added −5 to a whole *row* instead of a whole column. Softmax cannot see a constant added to an entire row, so the bias vanished. Section 2 shows exactly why.

This article is about making bugs like this one visible. They are the most common kind of PyTorch bug, and they are nearly impossible to spot by reading code. They become easy to find once you have an accurate picture of what a tensor is.

What you will learn

By the end of this article, you will be able to:

- predict the shape a broadcast produces, before running it;
- tell which operations share memory and which ones copy it;
- explain what `.backward()` computes, and why that is cheap;
- write a softmax that does not overflow;
- write a correct training loop and explain the order of its lines.

The article has six short sections, one idea each. It ends with a recap and a briefing for the programming exercise, in which you build a multi-head attention module step by step.

The running example: attention in one paragraph

We will practise on one function throughout. **Scaled dot-product attention** is the core of every transformer, and it appears in every module of this course. Here, it serves as a realistic function to practise tensor skills on. Do not worry if it is new: module 01 takes it apart properly.

The idea has four ingredients:

- each token asks a question, called its **query**;
- each token offers a label that describes it, called its **key**;
- each token carries some content, called its **value**;
- each token's output is an average of all the values, weighted by how well its query matches each key.

Written as a formula, with one query, key and value per row:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

Read it from the inside out. $QK^\top$ compares every query with every key using a dot product. Dividing by $\sqrt{d}$ keeps those scores in a sensible range. The softmax turns each row of scores into weights that add up to one. Multiplying by V takes the weighted average of the values.

Symbol	Meaning	Shape
T	number of tokens	a number
d	length of each query and key vector	a number
Q	the queries, one per row	$T \times d$
K	the keys, one per row	$T \times d$
V	the values, one per row	$T \times d_v$
output	one new vector per token	$T \times d_v$

The bias in the opening example was added to $QK^\top$, right before the softmax.

---

1. A tensor is a view onto a strip of memory

Let us start with how PyTorch stores numbers. Take a small matrix:

```
x = torch.arange(12).reshape(3, 4)
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

It looks like a grid, but the memory underneath is one flat strip of 12 numbers. PyTorch finds each element using a **stride**: the number of places to move along the strip to advance one position along a dimension.

```
memory:  [  0  1  2  3 |  4  5  6  7 |  8  9 10 11 ]
             row 0          row 1          row 2

x.stride()  ->  (4, 1)
```

```
    next row:     move 4 places
    next column:  move 1 place
```

So a tensor is really three things: a strip of memory called its **storage**, a shape, and a stride. That small fact explains most of the surprises in this section.

Transposing moves no data

Now transpose the matrix and look at the result:

```
x.t().stride()            # (1, 4)
x.t().is_contiguous()     # False
```

The transpose did not move a single number. It created a new tensor object that points at the same strip, with the two strides swapped. **Contiguous** means the strides follow plain row-by-row order; after the swap, they no longer do.

Many operations work this way. Slicing, `transpose`, `permute`, `view`, `expand` and `unsqueeze` all return a new view of the same memory. They cost almost nothing, however large the tensor is.

This design has three consequences that you will run into often.

Consequence 1: views share memory

If two tensors share storage, changing one changes the other:

```
x = torch.arange(12).reshape(3, 4)
y = x[0]        # a view of the first row
y += 100        # change y in place...
print(x)        # ...and x changes too
```

```
tensor([[100, 101, 102, 103],
        [  4,   5,   6,   7],
        [  8,   9,  10,  11]])
```

Sharing memory avoids copies, which makes it useful. It is also a common source of bugs. It is the reason PyTorch sometimes refuses in-place operations during training: changing a value in place would corrupt a value that autograd saved for later.

Consequence 2: `view` refuses where `reshape` copies

`view` only reinterprets existing memory. When the strides make that impossible, it raises an error instead of copying:

```
x.t().view(12)
```

```
RuntimeError: view size is not compatible with input tensor's size and
stride (...). Use .reshape(...) instead.
```

`reshape` does the same job, but quietly copies when it has to. Here it returns `[0, 4, 8, 1, 5, 9, ...]` in freshly allocated memory.

I prefer `view` while developing a model. A loud error at the exact line where the memory layout surprised you is worth more than a silent copy you did not know about.

Consequence 3: `expand` is free, `repeat` is not

Both of these calls turn a column of three numbers into a 3 × 4 matrix:

```
col = torch.tensor([[1.], [2.], [3.]])
col.expand(3, 4)    # stride (1, 0), storage of 12 bytes
col.repeat(1, 4)    # stride (4, 1), storage of 48 bytes
```

`expand` uses a stride of 0 along the new dimension, so it reads the same three numbers again and again. `repeat` makes real copies. The gap grows with size. A 1,024 × 64 tensor of 4-byte numbers occupies 256 KB, and expanding it into 32 copies still occupies 256 KB. Repeating it stores all 32, which is 8 MB. Module 05 meets this again in the memory cache used for text generation, where the difference reaches gigabytes.

> **In the interview:** if a model uses more memory than expected, look for `repeat`, `.contiguous()` and `reshape` calls that copy where a view would do.

---

2. Broadcasting follows one rule

Now we can explain the opening bug. When you combine tensors of different shapes, PyTorch **broadcasts**: it stretches the smaller tensor to match, without copying it. One rule decides how.

Line the shapes up from the right. Two sizes are compatible if they are equal, or if one of them is 1. A missing dimension counts as 1.

Here are the two versions of the bias, lined up:

```
correct:  scores    3 x 3
          key_bias      3     -> read as 1 x 3: one value per column

bug:      scores    3 x 3
          key_bias  3 x 1     -> one value per row
```

Both shapes follow the rule, so neither raises an error. The correct version adds −5 to column 1, which belongs to key 1. The buggy version adds −5 to row 1, which belongs to query 1:

```
correct: scores + key_bias      bug: scores + key_bias
[[ 0., -5.,  0.],               [[ 0.,  0.,  0.],
 [ 0., -5.,  0.],                [-5., -5., -5.],
 [ 0., -5.,  0.]]                [ 0.,  0.,  0.]]
```

Row 1 of the buggy version is now $[-5, -5, -5]$. Softmax divides each entry by the total of its row, so a constant added to the whole row cancels out:

$$\frac{e^{z_i-5}}{\sum_j e^{z_j-5}} = \frac{e^{-5}\,e^{z_i}}{e^{-5}\sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

That is the whole bug. The shapes were compatible, so broadcasting succeeded, and the softmax then hid the mistake completely.

Three habits that catch shape bugs

You cannot rely on error messages to catch these bugs. These three habits work instead:

1. Write shapes in comments, using letters: `B` for batch, `T` for tokens, `H` for heads and `D` for dimensions.
2. Assert shapes where functions start and end, for example `assert scores.shape == (B, H, T, T)`. It costs nothing and turns a silent mistake into a loud one.
3. Test with small sizes that are all different, such as `B=2, T=5, H=3, D=7`. If every size were 4, a swapped axis would go unnoticed.

`einsum` writes the shapes down for you

`torch.einsum` describes a tensor operation with index letters, the way you would on paper:

```
scores = torch.einsum("bhqd,bhkd->bhqk", q, k)
```

Read the string like this. For every batch `b` and head `h`, combine query position `q` with key position `k`, multiplying along the shared dimension `d` and summing it away. The letters after the arrow name the output axes. This computes exactly `q @ k.transpose(-2, -1)`; PyTorch confirms the two results are equal. The `einsum` version is easier to check against the maths, because the shapes are written in the string.

---

3. Autograd records what you do, then replays it backwards

PyTorch computes gradients for you. Let us look at the simplest possible case first:

```
w = torch.tensor(2.0, requires_grad=True)
loss = w * 3
loss.backward()
print(w.grad)        # tensor(3.)
```

The derivative of $3w$ with respect to w is 3, as expected. How did PyTorch get there?

The tape

When a tensor has `requires_grad=True`, PyTorch records every operation that uses it. Think of it as a tape recorder. Each recorded step remembers how to compute its own derivative, and which input values it needs to do so.

```
forward:    w  ---( * 3 )--->  loss
backward:   w.grad = 3.0  <---( * 3 )---  1.0
```

Calling `.backward()` plays the tape in reverse. Each step applies the chain rule and hands a gradient to the step before it. Together, the recorded steps form the **computational graph**.

What backward computes: vector–Jacobian products

This part confuses many people, so we will take it slowly.

A function from n numbers to m numbers has a **Jacobian**: an $m \times n$ matrix of partial derivatives. For a large model, these matrices would be enormous. Autograd never builds them.

Instead, backward receives a vector $\bar{y}$, with one entry per output. The bar is notation: it means "the gradient of the loss with respect to this", so $\bar{y}$ says how much the loss changes when each output changes. A barred quantity always has the shape of the quantity under the bar. Backward returns that vector multiplied by the Jacobian:

$$\bar{x}^\top = \bar{y}^\top J_f(x)$$

This is called a **vector–Jacobian product**, or VJP. In words, it answers "how does the loss change when each input moves a little?" without ever writing the Jacobian down.

Why is this the right design? In machine learning, the final output is a single number: the loss. So $\bar{y}$ is the number 1.0. One backward pass then gives the gradient for *every* parameter at once. It costs about as much as the forward pass, however many parameters the model has.

Two practical consequences follow:

- `.backward()` on a single number needs no argument, because PyTorch uses 1.0.
- `.backward()` on a tensor with several entries needs you to supply $\bar{y}$, as `gradient=`.

That is the core of autograd. The rest of this section covers four behaviours that follow from it.

Gradients add up until you clear them

Run the same backward pass twice, and the gradient doubles:

```
loss = w * 3; loss.backward()     # w.grad is 3
loss = w * 3; loss.backward()     # w.grad is now 6
```

PyTorch adds new gradients to `.grad` instead of replacing them. This is deliberate: it lets you add up gradients from several small batches before taking one update step. The price is that you must clear `.grad` before every training step, which is what `optimizer.zero_grad()` does. If you forget, the model still trains, slowly and badly, and nothing warns you.

The tape is thrown away after backward

To save memory, backward frees the values it recorded. So calling it a second time on the same graph fails:

```
w = torch.tensor(2.0, requires_grad=True)
loss = w * w
loss.backward()      # fine: w.grad is 4.0, since the derivative of w^2 is 2w
loss.backward()      # RuntimeError
```

```
RuntimeError: Trying to backward through the graph a second time (or
directly access saved tensors after they have already been freed). ...
```

The first call works and returns nothing, having written $2w = 4$ into `w.grad`. The second has no tape left to replay. Pass `retain_graph=True` to the first call if you really need a second one, and remember that the two gradients then add up to 8.

A related mistake causes what looks like a memory leak. If you collect losses with `losses.append(loss)`, every entry keeps its whole graph alive. Store `loss.item()` instead, which is a plain Python number.

`detach` and `no_grad` switch recording off

There are two ways to stop recording, and they work differently:

- `t.detach()` returns a tensor with the same data, cut off from the graph. Nothing computed from it is recorded.
- `with torch.no_grad():` stops recording for everything inside the block. Use it for evaluation.

If you call backward through a detached tensor, PyTorch reports that nothing was recorded:

```
b = torch.tensor(2.0, requires_grad=True)
d = (b * 2).detach() * 5
d.backward()
```

```
RuntimeError: element 0 of tensors does not require grad and does not
have a grad_fn
```

Only tensors you create get a `.grad`

Tensors you create directly, such as model weights, are called **leaf tensors**. Only leaves keep their gradient in `.grad` by default. Intermediate results do not, which saves memory. If you need the gradient of an intermediate result, call `.retain_grad()` on it before backward.

> **In the interview:** you may be asked "what does backward do?". A precise answer is: "It computes a vector–Jacobian product for each recorded operation, in reverse order, starting from 1.0 at the loss." This is short, correct, and explains why one backward pass is enough.

---

4. Numbers overflow, so write softmax carefully

Computers store each number in a fixed number of bits, so there is a largest number they can hold. Softmax hits that limit quickly, because it uses exponentials.

Here is softmax written straight from its definition, applied to three large scores:

```
z = torch.tensor([1000., 1001., 1002.])
z.exp()                          # tensor([inf, inf, inf])
z.exp() / z.exp().sum()          # tensor([nan, nan, nan])
```

e^{1000} is far too large to store, so it becomes infinity. Infinity divided by infinity is "not a number", written `nan`. This table shows how large z can get before e^z overflows:

Format	Largest number it can store	e^z overflows when z exceeds
float32	3.4×10^{38}	about 88.7
bfloat16	3.4×10^{38}	about 88.7
float16	65,504	about 11.1

The third column is the logarithm of the second, since e^z passes the limit exactly when z passes ln of it: $\ln(3.4 \times 10^{38}) = 88.7$ and $\ln 65{,}504 = 11.1$.

A score of 12 is not unusual. In float16, that alone is enough to break a naive softmax.

The fix: subtract the largest input first

Softmax gives the same answer if you subtract the same number from every input. That is the cancellation we saw in section 2. So subtract the largest input before exponentiating:

```
shifted = z - z.max()                    # tensor([-2., -1., 0.])
shifted.exp() / shifted.exp().sum()
```

```
tensor([0.0900, 0.2447, 0.6652])
```

Now the largest exponential is $e^0 = 1$, so nothing can overflow. The result matches `torch.softmax` exactly. Every serious softmax implementation uses this trick. Module 07 shows a streaming version of it, which is the key idea behind FlashAttention.

A common mistake: softmax before `cross_entropy`

A common mistake is to apply softmax to a model's output and then pass the result to `F.cross_entropy`. That function already applies softmax internally, so the softmax happens twice. Nothing crashes, but the model can never become confident. Here is the loss when the correct answer is class 0:

Model output (logits)	`cross_entropy(logits)`	`cross_entropy(softmax(logits))`
[2, 1, 0.1]	0.417	0.802
[10, 0, 0]	0.0001	0.552
[100, 0, 0]	0.0000	0.551

With the double softmax, the loss stops falling at 0.551, even when the model is certain. (For three classes, that floor is exactly $\ln(1 + 2/e)$.) So always pass the raw scores, called **logits**, to `cross_entropy`. Keeping the two steps in one function also lets PyTorch cancel the logarithm against the exponential exactly, instead of in rounded floating point.

---

5. `nn.Module` keeps track of your parameters

An `nn.Module` is a container for a model's layers and weights. When you assign a layer or a parameter to an attribute, the module **registers** it. Registration is how `.parameters()`, `.to(device)` and `.state_dict()` find everything.

A common mistake is to keep layers in a plain Python list:

```python
class Bad(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(4, 4), nn.Linear(4, 4)]

class Good(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([nn.Linear(4, 4), nn.Linear(4, 4)])
```

`Bad` reports 0 parameters; `Good` reports 40, which is two layers of a 4 × 4 weight matrix and a bias of 4. The forward pass of `Bad` works perfectly, so nothing looks wrong. But the optimiser never sees those layers, so they never learn. `nn.ModuleList` and `nn.Sequential` register their contents; a plain list does not.

Two more rules about registration are worth knowing:

- A tensor that should move with the model but should not be trained, such as an attention mask, needs `self.register_buffer("mask", mask)`.
- A plain tensor attribute is not registered at all, so it stays behind when the model moves to a GPU.

`eval()` and `no_grad()` do different jobs

These two are easy to confuse. This table shows what each one changes:

	Switches off dropout and freezes batch-norm statistics	Stops recording gradients
`model.eval()`	yes	no
`torch.no_grad()`	no	yes

You can check this directly. After `model.eval()`, dropout is off, so two calls give identical outputs. But those outputs still have `requires_grad=True`. Inside `torch.no_grad()`, they do not. For evaluation, you normally want both.

The training loop, line by line

Here is a standard training step. The order of the numbered lines matters.

```
for xb, yb in loader:
    xb, yb = xb.to(dev), yb.to(dev)
    loss = F.cross_entropy(model(xb), yb)
    opt.zero_grad()                      # 1. clear old gradients
    loss.backward()                      # 2. compute new ones
    nn.utils.clip_grad_norm_(
        model.parameters(), 1.0)         # 3. limit their size
    opt.step()                           # 4. update the weights
    sched.step()                         # 5. adjust the learning rate
```

Clear gradients before computing them. Limit their size after computing them and before updating the weights. If you swap any two of steps 1 to 4, the loop still runs but trains incorrectly. Step 3 is optional; it protects against a rare, very large gradient.

> **In the interview:** writing this loop from memory, and explaining why each line sits where it does, is a common warm-up question.

---

6. Speed comes from fewer, bigger operations

This section is short, because modules 05 and 09 cover performance in depth. One idea matters now. A GPU is fast at arithmetic but slow to start each operation and to move data around. So the fastest code runs a few large operations rather than many small ones.

The effect is large even on a CPU. Applying softmax to 512 rows one at a time, in a Python loop, took 5.0 ms on the laptop used to write this article. One call on the whole matrix took 0.16 ms, about 30 times faster, with an identical result. On a GPU the gap is usually wider.

Three rules follow from this:

- Operate on whole tensors, not on rows in a Python loop.
- Synchronise before timing GPU code. GPU operations run in the background, so a timer can stop before the work has finished. Call `torch.cuda.synchronize()` first; module 05 comes back to this.

- Train in `bfloat16` by default. It covers the same range as float32 (see the table in section 4), so it rarely overflows, and it uses half the memory.

One more fact matters for reproducibility. Adding floating-point numbers in a different order can give a slightly different result, and some GPU operations change that order between runs. `torch.use_deterministic_algorithms(True)` fixes the order, at some cost in speed. Results can still differ between different hardware, so compare them with a tolerance rather than exactly.

Recap

- A tensor is a strip of memory plus a shape and strides. Many operations create a new view of the same memory rather than a copy.
- Broadcasting lines shapes up from the right. A wrong shape often broadcasts successfully, so shape bugs rarely raise errors.
- Autograd records operations and replays them backwards as vector–Jacobian products. Gradients add up until you clear them.
- Softmax overflows unless you subtract the largest input first. Pass logits, not probabilities, to `cross_entropy`.
- A module trains only what it has registered. `eval()` and `no_grad()` do different jobs.
- Fast code uses a few large operations instead of many small ones.

The exercise: build multi-head attention, one skill at a time

What you will build. You will write a multi-head attention module: the same kind of `nn.Module` that sits inside every transformer. You get there in seven steps, and each step practises one idea from this article. At the end, PyTorch's own attention function checks your work. Keep your solution, because later modules build on it.

Time and setup. Allow two to three hours. Set up the environment once, from the course's root folder:

```
python -m venv .venv
.venv/Scripts/activate      # Linux or macOS: source .venv/bin/activate
pip install -r requirements.txt
```

How to run it. Open `00-pytorch-basics/exercise.ipynb` in Jupyter and run the cells in order. If you prefer a plain editor, `exercise.py` has the same content; run it with `python exercise.py` from inside the module folder.

How the checks work. Every place where you write code contains `raise NotImplementedError`. Replace that line with your implementation. Below it, a check cell tests your code and prints one line per check. Before you start, a check cell prints something like this:

```
[FAIL] step 1a  -- NotImplementedError:
0/1 passed.  Still failing: step 1a
```

Once your code is correct, the same cell prints:

```
[ok]   slice aliases
[ok]   transpose aliases
[ok]   expand aliases
```

```
[ok]   clone does not
[ok]   contiguous() on a transpose copies

All 5 checks passed.
```

A failing check never stops the notebook, so you can move on and come back later.

Step map. This table links each step to the part of the article it practises.

Step	You write	The idea it practises	Section
1	`shares_storage`; compare `expand` and `repeat`	tensors are views onto memory	1
2	`add_key_bias` and `causal_mask`	broadcasting	2
3	three `einsum` expressions	writing shapes explicitly	2
4	the softmax Jacobian and its VJP	what backward computes	3
5	`stable_softmax`	overflow	4
6	`attention`, then a study of the $\sqrt{d}$ scale	the running example	intro, 4
7	`MultiHeadAttention`, then a speed test	modules, buffers, batching	5, 6

One rule applies throughout: do not write a Python loop over a tensor dimension unless a step asks for one.

When you are done. Step 6 compares your `attention` with PyTorch's built-in version and confirms they match. Step 7 then checks that your mask is a registered buffer, that dropout respects `eval()`, and that no token can see the tokens after it.

Step 6b deserves a closer look, because it shows why the $\sqrt{d}$ matters. It measures the entropy of the attention weights, which tells you how spread out they are. With 64 tokens and $d = 512$, the entropy is 3.69 with the scaling and only 0.21 without it. (The maximum possible is 4.16.) Without the scaling, each token attends to almost a single other token.

If you get stuck.

- Print the shape of every tensor. Most mistakes in this exercise are shape mistakes.
- Run your function on a tiny input that you can check by hand, such as a 2 × 3 tensor.
- Reread the article section listed in the step map.
- If you are still stuck, open `solution.py` in this folder. It has one correct answer to every step, with comments explaining each choice. Compare your approach, not only your output.

Want more? The end of the notebook lists three stretch goals. They are writing your own backward pass with `torch.autograd.Function`, measuring how attention's memory grows with sequence length, and rewriting the module with the `einops` library.

Going deeper

Start here

- PyTorch, *Autograd mechanics* — https://pytorch.org/docs/stable/notes/autograd.html. The official explanation of the tape, in-place operations and `no_grad`. Read it after section 3.

Tensors and broadcasting

- PyTorch, *Broadcasting semantics* — https://pytorch.org/docs/stable/notes/broadcasting.html. The rule from section 2, with more examples.
- PyTorch, *Tensor views* — https://pytorch.org/docs/stable/tensor_view.html. The complete list of operations that return views instead of copies.
- Rogozhnikov (2022), *Einops*, ICLR — https://einops.rocks. A library for tensor operations with named axes. Worth trying once `einsum` feels natural.

Automatic differentiation, in depth

- Baydin, Pearlmutter, Radul & Siskind (2018), *Automatic Differentiation in Machine Learning: a Survey*, JMLR 18(153). arXiv:1502.05767. Section 3 explains forward and reverse mode clearly. Read it if you want the full picture behind section 3 of this article.
- Griewank & Walther (2008), *Evaluating Derivatives*, 2nd ed., SIAM. The standard textbook. Chapter 3 proves the cost results; chapter 12 covers checkpointing, which returns in module 04.
- Paszke et al. (2019), *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS. arXiv:1912.01703. Why PyTorch works the way it does, explained by the people who designed it.

Floating-point numbers and performance

- Goldberg (1991), *What Every Computer Scientist Should Know About Floating-Point Arithmetic*, ACM Computing Surveys 23(1). The classic explanation of rounding and overflow.
- Higham (2002), *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM. Chapters 1 to 4, for when you need full rigour.
- Micikevicius et al. (2018), *Mixed Precision Training*, ICLR. arXiv:1710.03740. How training in 16-bit formats works in practice.
- PyTorch, *CUDA semantics* — https://pytorch.org/docs/stable/notes/cuda.html. Why GPU operations run in the background, and how that affects timing.

The running example

- Vaswani et al. (2017), *Attention Is All You Need*, NeurIPS. arXiv:1706.03762. Section 3.2 defines the attention function used throughout this course.

Next: module 01 takes the attention function apart mathematically and derives its gradient by hand.